I'm assuming "4.6" means about 4.6 years of experience, which puts someone at a mid-level developer stage. In a banking project, that person usually works fairly independently on features, mentors juniors a little, and is trusted with production support.

A typical day

Morning

Check emails, Teams/Slack, and overnight alerts or production tickets (batch job failures, failed transactions, monitoring dashboards).
Attend the daily stand-up: share yesterday's progress, today's plan, and any blockers.
Review the sprint board (Jira) and pick up or update assigned stories.

Core development

Backend: Build and modify REST APIs and microservices (commonly Java/Spring Boot or .NET) for things like account services, fund transfers, payments, loans, KYC, and statements.
Write business logic, validations, and integrations with core banking systems, payment gateways (NEFT/RTGS/UPI/SWIFT), and third-party services.
Frontend: Build UI screens in Angular or React for internet banking, mobile banking, or internal staff portals. Handle forms, validations, state management, and API integration.
Database: Write and optimize SQL queries, stored procedures, and indexes (Oracle, SQL Server, PostgreSQL), and handle schema changes with migration scripts.
Messaging and batch: Work with Kafka or MQ for asynchronous processing, and with scheduled jobs for interest calculation, EOD processing, and reconciliation.

Quality and collaboration

Write unit and integration tests (JUnit, Mockito, Jasmine/Jest) and fix code-coverage gaps flagged by SonarQube.
Do peer code reviews, both giving and receiving feedback via pull requests.
Clarify requirements with business analysts, product owners, and testers.

Fix defects raised by QA or UAT.

DevOps side

Work with CI/CD pipelines (Jenkins, GitLab CI, Azure DevOps), Docker, and sometimes Kubernetes.
Support deployments across dev, SIT, UAT, and production environments, and join release or change-management calls.
Banking-specific responsibilities
Security: Follow secure coding practices (OWASP), handle authentication and authorization (OAuth2, JWT, MFA), encrypt sensitive data, and fix vulnerabilities from security scans.
Compliance and audit: Ensure audit logging, data masking, and adherence to regulations (PCI-DSS, GDPR, RBI/central bank guidelines), and keep documentation ready for audits.
Reliability: Banking systems need high availability and data consistency, so developers pay close attention to transaction management, idempotency, and error handling.
Production support: Analyze logs (Splunk, ELK, Kibana), debug incidents, and do root cause analysis, sometimes on a rotating on-call basis.
Sprint-level activities
Sprint planning and story estimation
Backlog grooming and refinement
Sprint demos and retrospectives
Technical design discussions and writing low-level design documents
What distinguishes the 4-5 year level
Owns features end to end, from design through deployment, with little supervision
Reviews code and guides junior developers
Contributes to design decisions and suggests performance or architecture improvements
Handles production issues with confidence

The exact tools vary by bank and project. If you tell me the tech stack (Java vs .NET, Angular vs React) or whether you're preparing for an interview, I can tailor this, for example with sample answers to "describe your day-to-day work."

Planning a contract-first SOAP service example for banking.

I'm assuming "xdl" means XSD (XML Schema Definition), since WSDL and XSD are the pair used together in SOAP contracts. Here is the concept first, then code for a banking example.

1. Spring MVC concept

Spring MVC follows the front-controller pattern.

Request flow:

The client sends a request to DispatcherServlet (the front controller).
HandlerMapping finds the matching controller method (@RequestMapping, @GetMapping, etc.).
The controller calls the service layer, which calls the repository/DAO.
The result is returned. With @RestController it is serialized to JSON by HttpMessageConverter; with @Controller a ViewResolver picks a view.
@ControllerAdvice handles exceptions globally.

Banking example (REST, Spring MVC):

java

```
@RestController
@RequestMapping("/api/v1/accounts")
@RequiredArgsConstructor
public class AccountController {

    private final AccountService accountService;

    @GetMapping("/{accountNo}/balance")
    public ResponseEntity<BalanceResponse> getBalance(@PathVariable String accountNo) {
        return ResponseEntity.ok(accountService.getBalance(accountNo));
```

```
    }

    @PostMapping("/transfer")
    public ResponseEntity<TransferResponse> transfer(@Valid @RequestBody
TransferRequest request) {
        return
ResponseEntity.status(HttpStatus.CREATED).body(accountService.transfer(request));
    }
}
java
public record TransferRequest(
    @NotBlank String requestId,          // idempotency key
    @NotBlank String fromAccount,
    @NotBlank String toAccount,
    @DecimalMin("1.00") BigDecimal amount) {}
java
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(InsufficientFundsException.class)
    public ResponseEntity<ErrorResponse> handle(InsufficientFundsException ex) {
        return ResponseEntity.status(HttpStatus.UNPROCESSABLE_ENTITY)
                .body(new ErrorResponse("INSUFFICIENT_FUNDS", ex.getMessage()));
    }
}
```

2. WSDL and XSD: the contract

XSD defines the data types (request/response structure).

WSDL defines the service contract: operations, messages, binding (SOAP), and endpoint URL. It imports or embeds the XSD.

Contract-first means you write the XSD first, then generate the WSDL and Java classes. Banks prefer this because the contract is stable and shared with other systems (core banking, partner banks).

Step 1: XSD (account.xsd)

xml

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           xmlns:tns="http://bank.com/schemas/account"
           targetNamespace="http://bank.com/schemas/account"
           elementFormDefault="qualified">

  <xs:element name="GetBalanceRequest">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="accountNumber" type="tns:AccountNumberType"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>

  <xs:element name="GetBalanceResponse">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="accountNumber" type="xs:string"/>
        <xs:element name="balance" type="xs:decimal"/>
        <xs:element name="currency" type="xs:string"/>
        <xs:element name="status" type="tns:AccountStatus"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>

  <!-- Validation rules live in the contract -->
  <xs:simpleType name="AccountNumberType">
    <xs:restriction base="xs:string">
      <xs:pattern value="[0-9]{10,16}"/>
    </xs:restriction>
```

```
  </xs:simpleType>

  <xs:simpleType name="AccountStatus">
    <xs:restriction base="xs:string">
      <xs:enumeration value="ACTIVE"/>
      <xs:enumeration value="DORMANT"/>
      <xs:enumeration value="BLOCKED"/>
    </xs:restriction>
  </xs:simpleType>
</xs:schema>
```

Step 2: WSDL (account.wsdl)

With Spring-WS you can auto-generate this from the XSD (see config below), but this is what it looks like:

xml

```
<wsdl:definitions xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
                  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
                  xmlns:sch="http://bank.com/schemas/account"
                  xmlns:tns="http://bank.com/schemas/account"
                  targetNamespace="http://bank.com/schemas/account">

  <wsdl:types>
    <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
      <xs:import namespace="http://bank.com/schemas/account"
schemaLocation="account.xsd"/>
    </xs:schema>
  </wsdl:types>

  <wsdl:message name="GetBalanceRequest">
    <wsdl:part element="sch:GetBalanceRequest" name="GetBalanceRequest"/>
  </wsdl:message>
```

```
  <wsdl:message name="GetBalanceResponse">
    <wsdl:part element="sch:GetBalanceResponse" name="GetBalanceResponse"/>
  </wsdl:message>

  <wsdl:portType name="AccountPort">
    <wsdl:operation name="GetBalance">
      <wsdl:input message="tns:GetBalanceRequest"/>
      <wsdl:output message="tns:GetBalanceResponse"/>
    </wsdl:operation>
  </wsdl:portType>

  <wsdl:binding name="AccountPortSoap11" type="tns:AccountPort">
    <soap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
    <wsdl:operation name="GetBalance">
      <soap:operation soapAction=""/>
      <wsdl:input><soap:body use="literal"/></wsdl:input>
      <wsdl:output><soap:body use="literal"/></wsdl:output>
    </wsdl:operation>
  </wsdl:binding>

  <wsdl:service name="AccountPortService">
    <wsdl:port binding="tns:AccountPortSoap11" name="AccountPortSoap11">
      <soap:address location="http://localhost:8080/ws"/>
    </wsdl:port>
  </wsdl:service>
</wsdl:definitions>
```

Step 3: Generate Java classes from the XSD (Maven, JAXB)

```
xml
<plugin>
  <groupId>org.codehaus.mojo</groupId>
  <artifactId>jaxb2-maven-plugin</artifactId>
  <executions>
```

```
      <execution>
        <id>xjc</id>
        <goals><goal>xjc</goal></goals>
      </execution>
    </executions>
    <configuration>
      <sources><source>${project.basedir}/src/main/resources/xsd/account.xsd</source></sources>
      <packageName>com.bank.account.ws</packageName>
    </configuration>
</plugin>
```

Dependency: spring-boot-starter-web-services. Note that on Spring Boot 3 / Jakarta EE you need the jakarta JAXB variants, so check your version.

Step 4: Endpoint (service implementation)

java

```
@Endpoint
@RequiredArgsConstructor
public class AccountEndpoint {

    private static final String NAMESPACE = "http://bank.com/schemas/account";
    private final AccountService accountService;

    @PayloadRoot(namespace = NAMESPACE, localPart = "GetBalanceRequest")
    @ResponsePayload
    public GetBalanceResponse getBalance(@RequestPayload GetBalanceRequest request) {
        Account acc = accountService.findByNumber(request.getAccountNumber());

        GetBalanceResponse response = new GetBalanceResponse();
        response.setAccountNumber(acc.getNumber());
        response.setBalance(acc.getBalance());
        response.setCurrency(acc.getCurrency());
```

```
        response.setStatus(AccountStatus.valueOf(acc.getStatus().name()));
        return response;
    }
}
```

Step 5: Configuration (exposes the WSDL automatically)

java

```
@EnableWs
@Configuration
public class WebServiceConfig extends WsConfigurerAdapter {

    @Bean
    public ServletRegistrationBean<MessageDispatcherServlet>
messageDispatcherServlet(ApplicationContext ctx) {
        MessageDispatcherServlet servlet = new MessageDispatcherServlet();
        servlet.setApplicationContext(ctx);
        servlet.setTransformWsdlLocations(true);
        return new ServletRegistrationBean<>(servlet, "/ws/*");
    }

    @Bean(name = "account")
    public DefaultWsdl11Definition wsdl11Definition(XsdSchema accountSchema) {
        DefaultWsdl11Definition wsdl = new DefaultWsdl11Definition();
        wsdl.setPortTypeName("AccountPort");
        wsdl.setLocationUri("/ws");
        wsdl.setTargetNamespace("http://bank.com/schemas/account");
        wsdl.setSchema(accountSchema);
        return wsdl;
    }

    @Bean
    public XsdSchema accountSchema() {
        return new SimpleXsdSchema(new ClassPathResource("xsd/account.xsd"));
```

```
    }
}
```

The WSDL is then available at http://localhost:8080/ws/account.wsdl.

Sample SOAP request and response

xml

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
                  xmlns:acc="http://bank.com/schemas/account">
  <soapenv:Body>
    <acc:GetBalanceRequest>
      <acc:accountNumber>1234567890</acc:accountNumber>
    </acc:GetBalanceRequest>
  </soapenv:Body>
</soapenv:Envelope>
```

3. How to explain this in an interview

"In our project, REST with Spring MVC served the Angular UI and mobile app, while SOAP with WSDL/XSD was used for integration with the core banking system and legacy partner systems."

"We followed contract-first: the XSD was agreed with the other team before coding, and JAXB generated the classes, so request validation (such as the account number pattern) was enforced by the schema itself."

"SOAP security came from WS-Security (UsernameToken or X.509 signatures) plus TLS."

Quick comparison for follow-up questions

	REST (Spring MVC)	SOAP (WSDL/XSD)
Format	JSON (usually)	XML only
Contract	OpenAPI/Swagger (optional)	WSDL + XSD (mandatory, strict)
Typical banking use	Channels: web, mobile	Core banking, B2B, legacy integrations
Standards	HTTP verbs, status codes	WS-Security, WS-ReliableMessaging

If you want, I can add a Spring RestTemplate/WebServiceTemplate client that calls this SOAP service, or a list of interview questions on SOAP vs REST with answers.

can you tell me a leger record api we need to auditor to check the account details of user and its send report to user explain me about the banking like goldman sachs internal tool project structure

I don't have inside knowledge of Goldman Sachs' internal tools (their real systems are not public), so what follows is a typical design for a bank's internal ledger and audit tool. It is the kind of structure you can credibly describe in an interview.

1. Core concepts

Ledger: the bank's book of record. Every money movement is stored as entries, never overwritten.

Double-entry: every transaction has a debit and a credit of equal amount (total debits = total credits).
Immutable / append-only: corrections are made by posting a reversal entry, never by editing or deleting a row.
Balance is derived from entries (or a verified running balance), which is what auditors check.

Auditor access is read-only, role-restricted, fully logged, and with sensitive fields masked.

2. Flow
The auditor logs in (SSO/OAuth2) with role AUDITOR.
The auditor searches an account's ledger entries by date range.
The auditor requests a report. The API returns 202 Accepted with a reportId.
A background job generates a PDF/CSV.
A Kafka event triggers the notification service, which emails the user a secure download link.
Every access is written to an audit log.
3. Project structure (Maven multi-module, Spring Boot)
ledger-audit-platform/
├── ledger-common/ # shared DTOs, exceptions, masking utils, constants

```
├── ledger-domain/                  # entities, repositories (JPA)
│   ├── LedgerEntry.java
│   ├── Account.java
│   ├── AuditLog.java
│   └── ReportRequest.java
├── ledger-service/                 # business logic
│   ├── LedgerQueryService.java
│   ├── ReportService.java
│   └── AuditLogService.java
├── ledger-api/                     # REST controllers, security config, Spring Boot main
│   ├── controller/
│   ├── security/
│   └── exception/
├── report-worker/                  # async report generation (PDF/CSV), Kafka consumer
├── notification-service/           # emails the user, Kafka consumer
├── db-migration/                   # Flyway scripts (V1__create_ledger.sql ...)
├── docker/ , k8s/                  # deployment configs
└── pom.xml (parent)
```

4. Database

```sql
CREATE TABLE ledger_entry (
  entry_id        BIGSERIAL PRIMARY KEY,
  transaction_id  VARCHAR(40)  NOT NULL,
  account_number  VARCHAR(20)  NOT NULL,
  entry_type      VARCHAR(6)   NOT NULL CHECK (entry_type IN ('DEBIT','CREDIT')),
  amount          NUMERIC(19,4) NOT NULL CHECK (amount > 0),
  currency        CHAR(3)      NOT NULL,
  running_balance NUMERIC(19,4) NOT NULL,
  description     VARCHAR(255),
  reversal_of     BIGINT REFERENCES ledger_entry(entry_id),
  posted_at       TIMESTAMP    NOT NULL DEFAULT now()
);
```

```
CREATE INDEX idx_ledger_acct_date ON ledger_entry(account_number, posted_at);

CREATE TABLE audit_log (
  id              BIGSERIAL PRIMARY KEY,
  auditor_id      VARCHAR(50) NOT NULL,
  action          VARCHAR(50) NOT NULL,
  account_number  VARCHAR(20),
  ip_address      VARCHAR(45),
  accessed_at     TIMESTAMP NOT NULL DEFAULT now()
);

CREATE TABLE report_request (
  report_id       UUID PRIMARY KEY,
  account_number  VARCHAR(20) NOT NULL,
  requested_by    VARCHAR(50) NOT NULL,
  from_date       DATE, to_date DATE,
  status          VARCHAR(15) NOT NULL,    -- REQUESTED, GENERATED, SENT, FAILED
  file_location   VARCHAR(255),
  created_at      TIMESTAMP DEFAULT now()
);
```

Also revoke UPDATE and DELETE on ledger_entry for the application DB user to enforce immutability.

5. API design

Method	Endpoint	Purpose
GET	/api/v1/audit/accounts/{accountNo}	Account details (masked)
GET	/api/v1/audit/accounts/{accountNo}/ledger?from=&to=&page=&size=	Paginated ledger entries
GET	/api/v1/audit/accounts/{accountNo}/reconcile	Check that sum of entries equals the balance
POST	/api/v1/audit/reports	Request a report, which is emailed to the user

GET /api/v1/audit/reports/{reportId} Report status

6. Code

Entity

java

```
@Entity
@Table(name = "ledger_entry")
@Getter
@NoArgsConstructor
public class LedgerEntry {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long entryId;
    private String transactionId;
    private String accountNumber;
    @Enumerated(EnumType.STRING)
    private EntryType entryType;              // DEBIT, CREDIT
    private BigDecimal amount;                // always BigDecimal for money
    private String currency;
    private BigDecimal runningBalance;
    private String description;
    private Long reversalOf;
    private LocalDateTime postedAt;
}
```

Repository

java

```
public interface LedgerEntryRepository extends JpaRepository<LedgerEntry, Long> {

    Page<LedgerEntry> findByAccountNumberAndPostedAtBetweenOrderByPostedAtDesc(
            String accountNumber, LocalDateTime from, LocalDateTime to, Pageable
```

```
pageable);

    @Query("""
        select coalesce(sum(case when e.entryType = 'CREDIT' then e.amount else
-e.amount end), 0)
        from LedgerEntry e where e.accountNumber = :acct""")
    BigDecimal computeBalance(@Param("acct") String accountNumber);
}
```

Controller (auditor-only)

```
java
@RestController
@RequestMapping("/api/v1/audit")
@RequiredArgsConstructor
@PreAuthorize("hasRole('AUDITOR')")
public class AuditController {

    private final LedgerQueryService ledgerService;
    private final ReportService reportService;

    @GetMapping("/accounts/{accountNo}/ledger")
    public Page<LedgerEntryDto> getLedger(@PathVariable String accountNo,
            @RequestParam LocalDate from, @RequestParam LocalDate to,
            @PageableDefault(size = 50) Pageable pageable, Principal auditor) {
        return ledgerService.getEntries(accountNo, from, to, pageable, auditor.getName());
    }

    @GetMapping("/accounts/{accountNo}/reconcile")
    public ReconcileResult reconcile(@PathVariable String accountNo, Principal auditor) {
        return ledgerService.reconcile(accountNo, auditor.getName());
    }
```

```
    @PostMapping("/reports")
    public ResponseEntity<ReportResponse> requestReport(
            @Valid @RequestBody ReportRequestDto request, Principal auditor) {
        UUID id = reportService.requestReport(request, auditor.getName());
        return ResponseEntity.accepted().body(new ReportResponse(id, "REQUESTED"));
    }
}
```

Service (with audit logging and masking)

```
java
@Service
@RequiredArgsConstructor
public class LedgerQueryService {

    private final LedgerEntryRepository ledgerRepo;
    private final AccountRepository accountRepo;
    private final AuditLogService auditLog;

    @Transactional(readOnly = true)
    public Page<LedgerEntryDto> getEntries(String acct, LocalDate from, LocalDate to,
                                           Pageable pageable, String auditorId) {
        auditLog.record(auditorId, "VIEW_LEDGER", acct);    // every access is logged
        return ledgerRepo
            .findByAccountNumberAndPostedAtBetweenOrderByPostedAtDesc(
                acct, from.atStartOfDay(), to.plusDays(1).atStartOfDay(), pageable)
            .map(LedgerEntryDto::from);
    }

    @Transactional(readOnly = true)
    public ReconcileResult reconcile(String acct, String auditorId) {
```

```
        auditLog.record(auditorId, "RECONCILE", acct);
        BigDecimal fromEntries = ledgerRepo.computeBalance(acct);
        BigDecimal recorded = accountRepo.findByAccountNumber(acct)
                .orElseThrow(() -> new AccountNotFoundException(acct)).getBalance();
        return new ReconcileResult(acct, fromEntries, recorded,
                fromEntries.compareTo(recorded) == 0);     // compareTo, not equals, for
BigDecimal
    }
}
```

Masking utility

```
java
public final class MaskUtil {
    public static String maskAccount(String acct) {
        return "XXXXXX" + acct.substring(acct.length() - 4);
    }
}
```

Report request: save, then publish a Kafka event

```
java
@Service
@RequiredArgsConstructor
public class ReportService {

    private final ReportRequestRepository reportRepo;
    private final KafkaTemplate<String, ReportEvent> kafka;

    @Transactional
    public UUID requestReport(ReportRequestDto dto, String auditorId) {
        ReportRequest r = new ReportRequest(UUID.randomUUID(), dto.accountNumber(),
```

```
                auditorId, dto.from(), dto.to(), "REQUESTED");
        reportRepo.save(r);
        kafka.send("report.requested", r.getReportId().toString(),
                new ReportEvent(r.getReportId(), dto.accountNumber()));
        return r.getReportId();
    }
}
```

Report worker: consume, generate, then notify

```
java
@Component
@RequiredArgsConstructor
public class ReportWorker {

    @KafkaListener(topics = "report.requested", groupId = "report-worker")
    public void generate(ReportEvent event) {
        // 1. fetch ledger entries for the period
        // 2. build PDF (iText / JasperReports) or CSV
        // 3. store in encrypted object storage (S3 with KMS)
        // 4. update status to GENERATED
        // 5. publish "report.generated" -> notification-service emails the user
    }
}
```

Notification service: email with an expiring link, not the file itself

```
java
@KafkaListener(topics = "report.generated", groupId = "notification")
public void notifyUser(ReportGeneratedEvent e) {
    String link = signedUrlService.createExpiringLink(e.getReportId(), Duration.ofHours(24));
    mailSender.send(e.getUserEmail(), "Your account statement",
```

"Your report is ready. This secure link expires in 24 hours: " + link);
}

7. Banking-grade concerns to mention

Security: SSO with MFA, RBAC, least privilege, TLS everywhere, field-level masking, encrypted storage.

Audit trail: who viewed what and when, stored in a tamper-evident table (restricted writes).

Data integrity: append-only ledger, reversals instead of edits, daily reconciliation job.

Reliability: idempotent event consumers (check reportId status before processing), retries and a dead-letter topic.

Compliance: data retention rules, GDPR/privacy controls, segregation of duties (the auditor cannot modify ledger data).

Performance: index on (account_number, posted_at), pagination, read replicas for audit queries so they don't hit the transactional database.

8. How to say it in an interview

"We built an internal audit platform where auditors could review account ledgers. The ledger is append-only with double-entry records. Auditors had read-only RBAC access through Spring Security, and every query was logged to an audit table. Report requests were asynchronous: the API published a Kafka event, a worker generated the PDF, and a notification service emailed the user a time-limited secure link. We ran a reconciliation check comparing summed ledger entries to the account balance, and I worked on the ledger query APIs, pagination and index tuning, and the Kafka consumers."